Research Statement

My research makes it possible for computers to come up with sets of control solutions for systems like cars, trains, and airplanes, that are mathematically guaranteed to be correct. Synthesizing control solutions is hard[1]. It requires reasoning about what the system must do *now* to stay safe *forever*, accounting precisely for environmental nondeterminism, interacting forces, and strategic control, over an uncountably infinite state space. My thesis *builds a framework* to tackle this problem using *logic/programming languages*, and soundly *scales synthesis* for practical cases using *intersymbolic algorithm design* (using AI in combination with formal methods), laying a foundation for new directions in safe autonomy.

Control Envelope Synthesis

Overview. Control envelopes are sets of control solutions. When developing a formally verified controller, (e.g., a train control system) one generally has to consider primary, mission-critical requirements, (e.g., safety) along with secondary or evolving requirements (e.g., smoothness of the ride). A possible approach to synthesizing a controller that is formally correct with respect to the primary requirements is: (1) first identify a maximal *set* of controllers formally verified to satisfy the primary requirement, i.e., a *verified control envelope* for the requirement, and then (2) optimize within this set for the controller choice that best meets the secondary requirements (e.g., using machine learning [3][2]). My thesis solves step 1, symbolic verified control envelope synthesis. It introduces the first approach applicable at the level of imperative program-like specifications[3], enabling synthesis for richly expressive control system models.

Example As a concrete example of what control envelope synthesis involves, consider a train controller that must stay within its *motion authority* (stop before some stopping points) to avoid collisions with other trains [7]. The controller can freely choose some engine acceleration or dynamic braking, but is not able to revise its decision for some bounded time latency (typically on the order of seconds) after its decision. A control envelope would answer: given the current state, what values of acceleration/braking are acceptable so that the train has a way to always remain safe? Any synthesis algorithm must account for all the competing influences that govern train motion. For example, uphill slope decreases velocity, which decreases resistance, which permits a more rapid increase in velocity, slope and curve resistance effects. Synthesis reasons *automatically* and precisely about these complex interactions that make it hard for a human to design a safe and efficient envelope, and even harder to ensure it is always safe.

The envelope is *symbolic*, supporting unbounded state space using variables that can represent any real number, and *parametric* in these variables (e.g., train weight w). The control envelope captures an entire *set* of control solutions, so in the same state, it may allow multiple different choices for acceleration or braking, and each choice leads to a different correct controller. Given a control

[1]The subject of an entire field, control theory.

[2]One way to use the control envelopes in combination with a machine learning (ML) based optimization step is to *shield* an ML controller agent's decisions. At training time, the agent is rewarded for staying within the envelope [11] and runtime, fallbacks are enforced when it tries to deviate from the verified set of solutions [3]. This allows behavior optimized by gradient descent within the safe solutions. Control envelopes can also statically verify a model [12].

[3]There is envelope synthesis work in the automaton community [1], though the input is at a different level of abstraction (automata, not program-like logic). I hope to investigate what ideas can transfer between the very different formalisms in the future.

system as input[4], how can we *represent* its control envelope, a potentially infinite control solution set, in a way that is *compositional* (synthesized/verified by the synthesis/verification of its parts)? How do we *define mathematically* what it means for this control envelope to be *correct*? How can such a definition lead to *automatic synthesis*? Are there ways to make the synthesis *scalable*? These are some of the questions my thesis answers, as summarized below.

Subvalue Maps The input to the control envelope synthesis problem is a *two-player game* between the controller and the environment [8, 5, 6]. The controller has some control moves, the environment adversarially resolves nondeterminism, and the controller's winning condition consists of meeting some critical control requirements. Input games are represented in an imperative program-like language, *differential game logic* (dGL) [10], that decomposes into subgames recursively along familiar syntactic structures: loops, sequential composition, and branching. As a symbolic, compositional representation of a control envelope, that we can reason about recursively along the syntactic structure of the game, my thesis proposes the *inductive subvalue map* [6], a map from subgames to formulas that hold exactly when it is acceptable to play the corresponding subgame. This map induces a nondeterministic policy for the controller: to check if it is acceptable to play into a subgame g in a given state, we only need to check that the subvalue associated with g holds in that state. We define a *correct* control envelope using recursive logical constraints on the subvalue formulas that ensure playing per *any* controller in the envelope guarantees that the controller player never gets stuck with no acceptable next action and can *always continue towards winning the game within the envelope*. We identify a backward precondition calculus that derives the subvalue map for a given dGL game and winning condition [6].

Scaling with LLMs The precondition calculus gives us a general, formally justified symbolic control envelope synthesis framework. But scalability remains an issue because of dependence on difficult subroutines like quantifier elimination (doubly exponential) [2], and invariant generation (undecidable, infinite search space). We attack the scalability bottlenecks in three different ways. (1) Invariant generation is performed by LLMs, leveraging their physics problem solving abilities, coupled with backtracking search. (2) Quantifier elimination and simplification are replaced with LLM queries, with soundness maintained by retrospective formal verification. (3) Verification can itself become a bottleneck, for which our solution is to have the LLM use an interactive theorem prover to break down the proof into tractable subgoals.

Control Envelope Synthesis via Angelic Refinement (CESAR). We identify another useful technique to synthesize for complicated games: *refinement*, or rewriting the game so that it is harder for the controller (maintaining soundness) but easier to symbolically reason about. For controllers that poll repeatedly in a single loop of some maximum time latency, we in fact identify systematic refinements via the CESAR algorithm [5]. For find that for controllers with *action permanence* (a hybrid analog of idempotence) and dynamics with polynomial solutions, CESAR is decidable.

Future Work

Control envelopes provide a practical path towards verifiably safe autonomy. They make it possible to safeguard controllers, even those based on legacy code or machine learning, by runtime monitoring against the verified envelope. Synthesis removes a large barrier to adoption: designing envelopes

[4]We consider a very general class of input control problems: those that can be expressed as hybrid games, in differential game logic [10].

in the first place. However, from my experience in the NSF I-Corps program, where I explored the commercialization of control envelopes by systematically interviewing industry stakeholders, I understand that some barriers remain. I would strategically choose research directions that target these barriers to industry adoption, accelerating research impact.

Formalizing for the Complexities of Reality. Formalization work with a similar flavor to my thesis will let control envelopes model increasingly complex real-world situations (partial observability, multiplayer coordination) with increasing scalability (efficiency improvements to the synthesis procedure for various problem classes, harnessing advances in AI via intersymbolic algorithms). How should we define probabilistic control envelopes that assure safety with some confidence interval given a probabilistic model of the environmental uncertainties? Extending the precondition calculus to follow the rules of stochastic reasoning [9] provides a path. How do we model control envelope synthesis for multi-agent, distributed systems where each agent may be open to collaborating? Statically agreed-upon contracts can be synthesized by treating all the collaborative control agents as a *single* controller. After carefully formalize the answer to these questions, how do we perform scalable synthesis for the new kinds of envelopes? The answer lies in careful algorithm design that combines the use of AI and verification.

Control Envelope Synthesis for the Masses. Reliable autoformalization tools can put envelopes within reach of audiences without formal methods expertise. Before synthesizing/verifying an envelope, one still needs a *formal model* of their system (the environment, control possibilities, and the desired guarantees). Today, writing such models is hard, a bottleneck for broader adoption of control envelopes, and more generally, formal methods. Meanwhile, potential users often already have the relevant information in a different form, e.g., design documents, log data, or simulation code. LLMs now show promise at writing formal models from informal natural language descriptions [4] (*autoformalization*). If we can use LLMs to *automatically* create formal models, with iterative feedback, users outside formal methods could access verified control envelopes and use them for downstream applications. Two flavors of challenges arise: making autoformalization *reliable*, e.g., with formal consistency checks and comparison to logs, and making it *usable*, e.g., with visual representations that help users spot errors.

Applications. Initial case studies demonstrate the benefits and costs of using control envelopes, and pave a path for future adopters. They can attract industrial collaborations and funding sources. During my Ph.D., I developed a verified *train* control envelope [7] under a contract with the *Federal Railroad Administration* (FRA) in collaboration with *Siemens*. Similar projects would be suitable for the FAA Aviation Research Grants program and the NSF Cyber-Physical System Foundations and Connected Communities program. Besides developing a state-of-the-art control envelope, the second part of such projects is the *end application*. I am currently collaborating with cybersecurity researchers at University of Utah to develop a control envelope-based tool to *analyze designs* of systems like water treatment plants for their safety and infrastructure wear costs when mitigating cyber attacks. Another future application is, when *fine tuning foundation models* for robot control using reinforcement learning (RL), feedback on the model's control decisions should reason over unbounded time to catch a mistake now even if its consequences won't be seen till far in the future. Control envelopes batch-compute the solutions for the entire control problem over infinite time once and for all, so that later checking a single action for RL feedback is efficient.

At X University, I would be excited to work with students and collaborators to push the control envelope per this vision, and more broadly, contribute towards safe autonomy.

References

[1] Benerecetti, M., Faella, M.: Automatic synthesis of switching controllers for linear hybrid systems: Reachability control. ACM Trans. Embed. Comput. Syst. **16**(4) (May 2017). https://doi.org/10.1145/3047500

[2] Davenport, J.H., Heintz, J.: Real quantifier elimination is doubly exponential. J. Symb. Comput. **5**(1/2), 29–35 (1988). https://doi.org/10.1016/S0747-7171(88)80004-X

[3] Fulton, N., Platzer, A.: Safe reinforcement learning via formal methods: Toward safe control through proof and learning. AAAI 2018. https://doi.org/10.1609/aaai.v32i1.12107

[4] Kabra, A., Laurent, J., Bharadwaj, S., Martins, R., Mitsch, S., Platzer, A.: Can large language models autoformalize kinematics? FMCAD 2025. https://doi.org/10.34727/2025/isbn.978-3-85448-084-6_13

[5] Kabra, A., Laurent, J., Mitsch, S., Platzer, A.: CESAR: Control envelope synthesis via angelic refinements. TACAS 2024. https://doi.org/10.1007/978-3-031-57246-3_9

[6] Kabra, A., Laurent, J., Mitsch, S., Platzer, A.: Hybrid game control envelope synthesis (2025). https://doi.org/10.48550/arXiv.2508.05997

[7] Kabra, A., Mitsch, S., Platzer, A.: Verified train controllers for the federal railroad administration train kinematics model: Balancing competing brake and track forces. IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. **41**(11), 4409–4420 (2022). https://doi.org/10.1109/TCAD.2022.3197690

[8] Nerode, A., Yakhnis, A.: Modelling hybrid systems as games. CDC 1992. https://doi.org/10.1109/CDC.1992.371272

[9] Platzer, A.: Stochastic differential dynamic logic for stochastic hybrid programs. CADE 2011. https://doi.org/10.1007/978-3-642-22438-6_34

[10] Platzer, A.: Differential game logic. ACM Trans. Comput. Log. **17**(1), 1:1–1:51 (2015). https://doi.org/10.1145/2817824

[11] Qian, M., Mitsch, S.: Reward shaping from hybrid systems models in reinforcement learning. NFM 2023. https://doi.org/10.1007978-3-031-33170-1_8

[12] Teuber, S., Mitsch, S., Platzer, A.: Provably safe neural network controllers via differential dynamic logic. NeurIPS 2024. https://doi.org/10.5555/3737916.3737967